The Last Percent: Concept Emergence and the Jump from Approximation to Solving in Neural CSPs

Anonymous Author(s)
Affiliation
Address
email

Abstract

Neural solvers for combinatorial optimization often achieve strong approximation ratios while still failing to produce valid solutions. We study this approximation–solving gap through the lens of *learned concepts*: solver-relevant properties encoded in the latent space of a GNN. Using random SAT as a case study, we apply a controlled sequence of modifications to OPTGNN, a Semi-Definite-Programming (SDP)-guided graph neural network. Each modification is inspired by classical heuristic design: (i) a single recurrent message-passing unit run for more iterations; (ii) an explicit rounding component paired with a logit-space loss; and (iii) constraint-level aggregation inspired by factor graphs.

Our representation analysis shows that high approximation ratios arise across all variants, but solving emerges in variants that develop the concept of *confidence*: a signal encoding how likely a variable's current assignment is to agree with an optimal solution. Models that have solving capacity encode this confidence geometrically, via the classical notion of *support*, and preferentially revise low-confidence variables; approximating models do neither. These results suggest that solving is not a smooth extension of approximation, but a qualitatively distinct capability, and provide concept-level criteria for designing future GNN-based solvers.

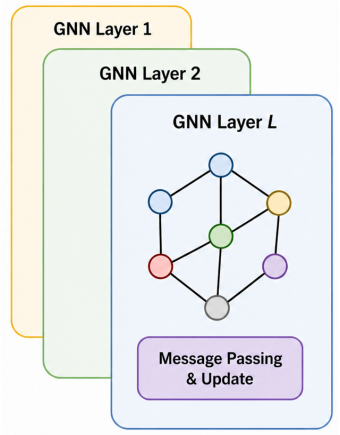
1 Introduction

A common recipe for improving neural models is to make them larger, run them longer, or tune the objective: add layers, increase inference iterations, or refine the loss to better match the desired output. In this work, we show that for Graph Neural Networks (GNNs) applied to combinatorial optimization, this intuition can be misleading. Rather than a smooth improvement curve, we find a *singular transition*: models move abruptly from high-quality approximate solutions, often satisfying 99% of the constraints, to fully solving instances, satisfying 100%. This last percentage point is not a marginal gain; it marks a qualitative architectural shift from approximation to exact feasibility.

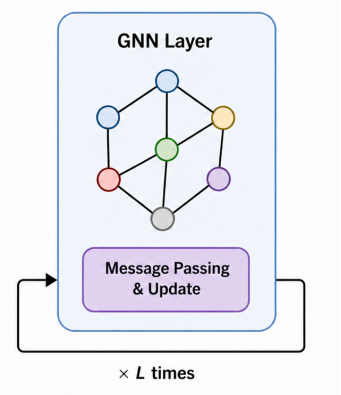
We identify this transition point as the model's ability to learn a specific class of solver-relevant properties. In particular, we identify *variable confidence*—a measure of how strongly a variable's current assignment agrees with an optimal assignment. As it turns out, this concept can be computed locally in some cases, making it feasible for GNNs to learn. This was shown in the pre-ML literature studying random CSPs (Constraint Satisfaction Problems) such as SAT and k-colorability Alon and Kahale [1994], Flaxman [2008]. When this concept is absent, models achieve strong approximation ratios but fail to solve. When it emerges, the GNN architecture exhibits solving capacities.

We demonstrate this phenomenon on two canonical constraint satisfaction problems: *Boolean Satisfiability (SAT)* and a hypergraph variant of Max-Cut, namely *hypergraph 2-coloring*, also known as *NAE-3SAT*. Our starting point is OPTGNN Yau et al. [2024], an acclaimed general-purpose GNN

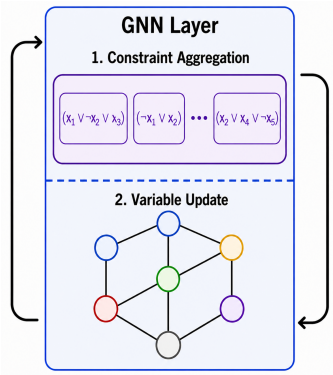
Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.



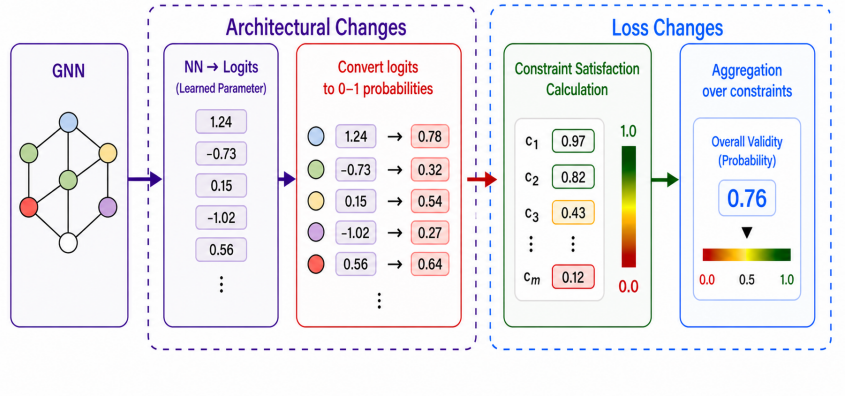
(a) Stacked architecture.



(b) Shared recurrent layer.



(d) Clause-level aggregation.



(c) Logit-space readout/loss.

Figure 1: Controlled OptGNN variants and their architectural modifications. Panels (a)–(c) show the three incremental components studied in this work: (a) stacked layer-specific GNN updates, (b) a shared recurrent GNN layer applied for L iterations, and (c) a logit-space readout/loss that maps variable logits to probabilities, evaluates clause validity, and aggregates the resulting constraint-level signal. Panel (d) shows the clause-level aggregation variant, which explicitly aggregates constraint representations before feeding the result back into the variable update. Together, these panels define the progression from stacked message passing, to recurrent refinement, to logit-space supervision, and finally to explicit clause-level aggregation.

framework for unsupervised combinatorial optimization, derived from Semi-Definite Programming (SDP)-based continuous relaxations. These two settings align with the SAT and Max-Cut problems studied in OptGNN, allowing us to examine the approximation-to-solving transition within the same relaxation-based framework.

We introduce a sequence of controlled modifications inspired by classical heuristic design principles, illustrated in Figure 1. Starting from the original model, we consider three composable changes: (1) replacing the fixed multi-layer update sequence with a single recurrent message-passing layer run for more iterations, analogous to running an iterative heuristic longer; (2) introducing a learned *rounding hyperplane*, which projects the continuous embeddings toward a one-dimensional assignment coordinate before the loss is evaluated, mimicking the rounding step that converts an SDP solution into a discrete assignment; and (3) augmenting the architecture with explicit constraint-level aggregation through a bipartite variable–constraint structure, aligned with the classical factor-graph view of constraint systems Pearl [1988]. These modifications allow us to isolate which ingredients merely improve approximation quality and which are necessary for the transition to solving. Crucially, while each component contributes to performance, combining all components yields the strongest and most robust singular transition from near-feasibility to exact feasibility.

This perspective suggests a shift in how progress in neural combinatorial optimization should be evaluated. Rather than comparing architectures solely by approximation ratios or benchmark performance, we argue that models should be compared through the *concepts they learn*. This provides a unified lens for understanding when and why a model transitions from approximation to solving, and offers a principled direction for designing more effective neural solvers.

We use two simple insights to formalize this distinction.

Insight 1: The logit loss exposes a confidence signal. For a SAT clause C, let $x_i \in [0, 1]$ denote the relaxed truth value of variable i. The relaxed clause-unsatisfaction loss is

$$u_C^{\mathrm{SAT}}(x) = \prod_{i \in C^+} (1 - x_i) \prod_{i \in C^-} x_i.$$

This loss is a product of false-literal indicators. Therefore, for near-discrete assignments, it induces a strict ordering over local clause patterns:

$$u_C(TTT) < u_C(TTF) < u_C(TFF) < u_C(FFF).$$

Thus, among satisfied clauses, TFF is closest to violation: the clause is satisfied, but only one literal prevents it from becoming false. Unlike FFF, which gives a correction signal, TFF gives a confidence signal by identifying the literal that preserves satisfaction. Variables that participate in many such clauses are therefore locally identifiable as confidence-critical variables. We formalize this in Appendix A, Insight 1.

Insight 2: the SDP loss mixes confidence with high-dimensional geometry. Let V denote the high-dimensional embedding matrix used by the SDP-relaxation OPTGNN. The clause objective in Yau et al. [2024] is

$$\Phi_C(V) = \frac{1}{8} \left(7 + a_i(V) + a_j(V) + a_k(V) - b_{ij}(V) - b_{ik}(V) - b_{jk}(V) + c_{ijk}(V)\right),$$

where the linear terms a_i, a_j, a_k are inner products between variable embeddings and a fixed reference vector v_0, while the remaining terms are inner products among the variable embeddings themselves.

Thus, while the logit loss separates patterns such as TFF at the level of relaxed clause values, the SDP objective expresses the same distinction through high-dimensional vector moments. These inner-product terms vary across clauses and embeddings, introducing geometric noise that can obscure the confidence signal. We surmise that the logit augmentation reduces this noise by learning a single rounding direction, making the confidence signal easier to isolate. We formalize this intuition in Appendix A, Insight 2.

Our contributions are as follows:

- We expose a sharp approximation–solving gap in relaxation-based GNNs through a controlled sequence of OPTGNN variants.
- We show that closing this gap is associated with the model learning a confidence signal, instantiated in SAT as the classical notion of *support*.
- We demonstrate that standard architectural improvements—more iterations, learned rounding, or modified losses—can improve approximation but do not by themselves reliably induce solving.
- We identify a concrete architectural ingredient, explicit constraint-level aggregation, that enables more coherent confidence formation and yields the strongest transition to solving.
- We outline a broader design principle for neural combinatorial optimization: architectures should not only optimize relaxed objectives, but explicitly support the emergence of solver-relevant concepts.

2 Background and Related Work

Constraint systems, assignments, and confidence. Yau et al. [2024] studies GNNs for discrete constraint systems in which the output is an assignment satisfying local constraints. We focus on two such settings: SAT and NAE-SAT. NAE-SAT can be viewed as a hypergraph variant of Max-Cut, expressed in SAT terminology: each clause is satisfied only if it contains both a true and a false literal.

Throughout, we use *constraint* as the generic term and *clause* when referring specifically to SAT-type formulas Cook [1971]. Given a formula F and an assignment ϕ, the *support* of a variable x in SAT is the number of clauses uniquely satisfied by x:

$$\mathrm{support}_\phi(x) = \left|\{C \in F : C \text{ is satisfied under } \phi \text{ only by } x\}\right|. \tag{1}$$

For NAE-SAT, we use the analogous definition: x supports a clause if its literal is the "odd one out" under ϕ, i.e., the only true literal or the only false literal in the clause. Equivalently, flipping x would make all literals in that clause equal and would violate the NAE constraint.

Support is a classical solver-relevant quantity. Survey Propagation uses related signals to identify biased variables whose assignments are highly constrained Mézard and Zecchina [2002], Achlioptas and Ricci-Tersenghi [2006]; backbone analyses show that highly constrained variables often take the same value across satisfying assignments Nudelman et al. [2004], Achlioptas and Coja-Oghlan [2008]; and WalkSAT-style heuristics implicitly prefer flipping low-support variables to escape local minima Lincoln and Yedidia [2020]. In all cases, support acts as a confidence signal: it measures how strongly the current assignment depends on a variable.

Operational definition of a learned concept. In this paper, a *concept* is not used informally. We say that a GNN has learned a concept $g_\phi(v)$ if that property is reliably decodable from the latent embedding $h_v^{(t)}$ by a simple probe, typically a linear map, across held-out instances:

$$\widehat{g}_\phi(v) = w^\top h_v^{(t)} + b. \tag{2}$$

Thus, the insight that a model learns confidence is falsifiable: it fails if support, or the relevant confidence quantity, cannot be predicted from the embeddings above the stated probe baseline on unseen instances. This operationalization allows us to compare model variants by both performance and the extent to which their latent spaces encode solver-relevant confidence.

Approximation vs. solving. Approximation and solving are related but distinct goals. In random 3-SAT, a uniformly random assignment already satisfies each clause with probability $7/8$, so strong approximation is achievable from the outset. Closing the remaining gap to 100%, however, is qualitatively harder: Håstad's inapproximability result shows that improving beyond the $7/8$ threshold is NP-hard in the worst case Håstad [2001]. Moreover, for random SAT above certain densities, assignments satisfying most clauses may still be far in Hamming distance from any satisfying assignment, so repairing the final violations can require coordinated changes across many variables Achlioptas and Coja-Oghlan [2008]. Continuous relaxation-based objectives amplify this gap: they reward incremental score improvements without enforcing discrete feasibility, allowing high objective values while remaining far from a valid solution.

Related work: GNNs and SDP-based neural solvers. GNNs update variable representations by repeatedly aggregating information from neighboring variables or constraints Gilmer [2017], Zhou et al. [2020]. This local message-passing structure is well suited to combinatorial problems such as SAT and graph coloring; our focus is what happens when it is paired with a relaxation objective. OPTGNN Yau et al. [2024], amongst others Tönshoff et al. [2021], is representative of this setting: it combines iterative graph-based updates with an SDP-inspired objective, refining variable representations using both relational structure and relaxation gradients. Related neural solvers fall into two broad groups. Direct neural solvers such as NEUROSAT Selsam et al. [2019] and GNN-GCP Lemos et al. [2019] show that message passing can encode rich combinatorial structure, but they are not tied to the same optimize-then-round relaxation pipeline. Unsupervised relaxation-based solvers Tönshoff et al. [2021], Yau et al. [2024], by contrast, emphasize scalability and generality across problem classes, but sharpen the question of what additional structure is needed to move from strong approximation to actual solving.

Recent interpretability work shows that confidence-related concepts such as *support* or degree can appear in neural solvers Shoham et al. [2024, 2026], but it remains unclear when they form, why some models acquire them and others do not, and how they translate into exact solving. Our work addresses this gap by linking objective design, architecture, representation, and solving behavior.

3 Methodology

We study the transition from approximation to solving through a controlled sequence of modifications to an SDP-inspired neural solver. Figure 1 summarizes the four variants. OPTGNN is the original relaxation-driven model; OPTGNN-IR replaces the original multi-layer update sequence with a single recurrent update map applied over multiple iterations; OPTGNN-LOGIT adds a fully connected layer ($d \to 1$) and passes it to a sigmoid that 'rounds' the high-dimensional latent space to the interval [0,1], intuitively translating the embedding of variable x_i to the probability that x_i takes the value True (like in Belief Propagation). Additionally, in OPTGNN-LOGIT, we replace the SDP-style training objective with the logit-space clause-unsatisfaction loss in Eq. (4), while keeping the gradient-based update dynamics unchanged. Finally, OPTGNN-CLAUSE adds explicit clause-level aggregation and is trained with the same logit-space objective. This progression lets us isolate the roles of recurrence, objective design, and architectural alignment with the SAT factor graph.

Base model. Our starting point is OPTGNN Yau et al. [2024], a graph neural architecture coupled to a continuous SDP-inspired relaxation. Given a problem instance, the model maintains a vector representation for each variable, updates these representations using gradients of the relaxation objective, and obtains the final assignment by thresholding the final variable states:

$$v_{i+1} = \mathrm{Norm}(M_i(v_i, \nabla_i \mathrm{Norm}(L(v_i)))), \qquad \mathrm{out} = \mathrm{sign}(v_{i+1} \cdot \mathit{random\_vector}). \tag{3}$$

Here, L is the SDP-based objective, M_i is the learned update map at iteration i, and $\mathrm{Norm}(\cdot)$ stabilizes the dynamics. In the original OPTGNN, this corresponds to an untied sequence of update maps $M_1, \ldots, M_{32}$; preliminary experiments showed that using deeper sequences (e.g. $M_1, \ldots, M_{100}$), suffered from vanishing gradients leading to worse results. Following Yau et al. [2024], for each instance we sample a single random projection vector and use it for all variables when producing the final thresholded assignment; this vector is fixed throughout the inference run.

Controlled modifications. Starting from OPTGNN, we study three cumulative modifications:

- **Iterative refinement (OPTGNN-IR):** we replace the original sequence of step-specific maps $M_1, \ldots, M_{32}$ with a single shared recurrent map M, which is applied for 32 iterations during training, and up to 500 in inference. This isolates the effect of repeated computation with tied parameters.
- **Logit-space loss (OPTGNN-LOGIT):** we map each variable state v_i to a binary logit x_i and replace the SDP objective with an aggregate of relaxed clause-unsatisfaction terms. In what follows C^+ and C^- denote the sets of positive and negative literals in clause C, respectively. The term $u_C(x)$ measures the degree to which clause C is unsatisfied: when assignments are discrete $x_i \in \{0, 1\}$, we have $u_C(x) = 1$ if and only if C is unsatisfied:

$$\begin{aligned} u_C^{\mathrm{SAT}}(x) &= \prod_{i \in C^+} (1 - x_i) \prod_{i \in C^-} x_i, \qquad u_C^{\mathrm{NAE}}(x) = u_C^{\mathrm{SAT}}(x) + u_C^{\mathrm{SAT}}(1 - x), \\ \mathcal{L}_{\mathrm{logit}}^{\mathrm{SAT}}(x) &= \sum_{C \in F} u_C^{\mathrm{SAT}}(x), \qquad \mathcal{L}_{\mathrm{logit}}^{\mathrm{NAE}}(x) = \sum_{C \in F} u_C^{\mathrm{NAE}}(x). \end{aligned} \tag{4}$$

- **Constraint-level aggregation (OPTGNN-CLAUSE):** we introduce clause representations $c_j = \mathrm{Norm}\Big(M'\Big(\sum_{v_i \in C_j} v_i\Big)\Big)$, and $v_{i+1} = \mathrm{Norm}(M(v_i, \nabla_i, \sum_{C_j \ni v_i} c_j))$ and feed them back to variables through the update map together with the gradient signal. This gives the model direct access to joint constraint structure, e.g., which variables uniquely sustain satisfied clauses.

Evaluation. For each model we report two primary metrics. The *approximation ratio* is the fraction of clauses satisfied by the predicted assignment, averaged over instances; a ratio of 1 corresponds to exact solving. The *solving rate* is the fraction of instances for which the predicted assignment satisfies all clauses. These two metrics jointly characterize the approximation–solving gap: a model may achieve a high approximation ratio while its solving rate remains low.

We complement these with a Hamming-distance analysis, measuring how far a high-approximation failed assignment is from a satisfying solution. For each failed instance, we compute the normalized distance to the nearest satisfying assignment found by exhaustive search. Since this is substantially more expensive than satisfiability checking, we use smaller instances ($n = 250$, 100 failed instances at $c = 3.7$); the main benchmark uses $n = 1000$.

Data and evaluation protocol. For SAT, we follow the data generation procedure of Yau et al. [2024]. Random 3-SAT instances are generated by sampling $m = cn$ clauses uniformly, each containing three randomly selected variables with independently sampled literal signs. Training uses 2K instances with $n = 100$ variables and clause density $c \sim \mathcal{U}[3.5, 3.9]$, near the satisfiability phase transition at $c \approx 4.26$ Achlioptas et al. [2005], Friedgut et al. [1999]. Evaluation uses larger random instances with $n = 1000$ across clause densities $c \in \{0.5, 1.0, 2.0, 2.5, 3.0, 3.5, 3.7, 3.9, 4.1, 4.2, 4.3\}$.

For NAE-SAT, we generate random instances analogously, but evaluate clauses under the not-all-equal constraint. Training uses random instances with $n = 100$ and clause density $c \in [0.5, 1.0, 1.5, 1.8, 2.0, 2.1, 2.5]$, near the NAE-SAT phase transition at $c \approx 2.11$ Achlioptas et al. [2001]. Evaluation uses larger random instances generated analogously to the SAT setting.

As c increases, the geometry of the solution space undergoes structural transitions that make instances progressively harder: the solution space fragments from a single connected cluster into exponentially many well-separated clusters, local search becomes trapped, and eventually satisfying assignments cease to exist beyond the threshold Mézard and Zecchina [2002], Mertens et al. [2006], Achlioptas and Coja-Oghlan [2008], Braunstein et al. [2005], Seitz et al. [2005], Braunstein et al. [2006].

Representation analysis. Let $\mathbf{E} \in \mathbb{R}^{n \times d}$ denote the matrix of final variable embeddings. The final assignment is extracted from the model output itself: for OPTGNN and OPTGNN-IR, we use the sign of the randomized projection, while for OPTGNN-LOGIT and OPTGNN-CLAUSE, we threshold the sigmoid output at 0.5. Support, defined in Section 2, is then computed for each variable under this induced assignment.

To evaluate how directly support is encoded in the learned representations, we test whether it is linearly decodable from the embedding space. Specifically, for each variable we use its final embedding vector $\mathbf{E}_i \in \mathbb{R}^d$ as input to a linear SVM and train classifiers to separate adjacent support levels $(0, 1)$, $(1, 2)$, and $(2, 3)$. We report the mean 5-fold Macro-F1 score across these adjacent pairs. Importantly, this probe is a test of simple linear accessibility: weak capacity to separate support levels does not imply that support is absent from the representation, but rather that it is not encoded in an easily linearly decodable form.

To measure whether the model *uses* support during inference, to prefer flipping low-support variables, we compute support-conditioned flip probabilities. At each iteration, we extract the Boolean assignment from the model output and record whether each variable changes value between consecutive iterations. For each support level s, we normalize by the number of variables with that support: $p_{\text{flip}}(s) = \frac{\#\text{ flipped variables with support } s}{\#\text{ variables with support } s}$. We then average these probabilities over iterations and instances, ensuring that frequent support levels do not dominate rarer ones.

4 Results

We modify the released Yau et al. [2024] implementation as described in Section 3, keeping all other settings unchanged. Inference uses 32 iterations for OPTGNN and 100 for the other variants, with hidden dimension 32 and final sign decoding. Experiments used Python 3.11 and PyTorch on an AMD Ryzen 9 5950X workstation with 32 GB RAM and an NVIDIA RTX 3080. Each model was trained and evaluated over 10 independent runs, each taking at most 40 minutes; across runs, reported metrics varied by standard deviation at most 0.02.

From approximation to solving. Table 1 (and Table 4 in the Appendix) reveals a clear progression across model variants as clause density increases. In the sparse regime ($c \leq 1$), all variants achieve high solving rates, with all augmented variants solving every instance exactly and the base OPTGNN solving most instances. As density grows into the moderate range ($1 < c \leq 3$), the base OPTGNN collapses to zero solving rate while maintaining a high approximation ratio around 0.98–0.99, exposing the approximation–solving gap. Iterative refinement alone does not close this gap: OPTGNN-IR

solves all sparse instances but also collapses once density increases. The logit-space loss substantially improves solving, with OPTGNN-LOGIT solving all instances up to $c \leq 2.5$ and retaining partial solving capacity at $c = 3.0$ and $c = 3.5$, before collapsing at higher densities. The full model, OPTGNN-CLAUSE, achieves the strongest performance, solving all instances up to $c = 3.0$ and maintaining non-trivial solving rates through $c = 4.1$, including $c = 3.9$—the density at which NEUROSAT, a supervised GNN, is reported to fail Selsam et al. [2019], Shoham et al. [2024]. Crucially, approximation ratios remain uniformly high (≥ 0.97) across all variants and densities, confirming that the gains are in solving capacity rather than approximation quality.

Table 1: SAT results on random SAT test instances for different c values with $n = 1000$, using 100 independently generated instances per density. Each cell reports solving rate | approximation. The standard deviation across test instances was below 0.01 in all reported settings and is therefore omitted for readability. Approximation remains high across variants, while solving improves sharply only after the logit-space loss and clause-level aggregation.

Model	$0.5 - 1.0$	$2.0 - 2.5$	3.0	3.5	3.7	3.9	≥ 4.1
OPTGNN	.92|.99	.00|.99	.00|.98	.00|.98	.00|.98	.00|.97	.00|.97
OPTGNN-IR	1.0| 1.0	.00|.99	.00|.98	.00|.98	.00|.98	.00|.98	.00|.98
OPTGNN-LOGIT	1.0| 1.0	1.0| 1.0	.83|.99	.09|.99	.00|.99	.00|.99	.00|.99
OPTGNN-CLAUSE	1.0| 1.0	1.0| 1.0	1.0| 1.0	.93|.99	.64|.99	.22|.99	.01|.99

High approximation need not imply closeness to a solution. On failed random SAT instances with $n = 250$ and $c = 3.7$, the baseline reaches about 0.98 approximation but remains roughly 10% away, in normalized Hamming distance, from the closest satisfying assignment found by exhaustive exact search. Thus, high objective value does not mean the model is a few flips from feasibility; the remaining gap can require coordinated changes across many variables.

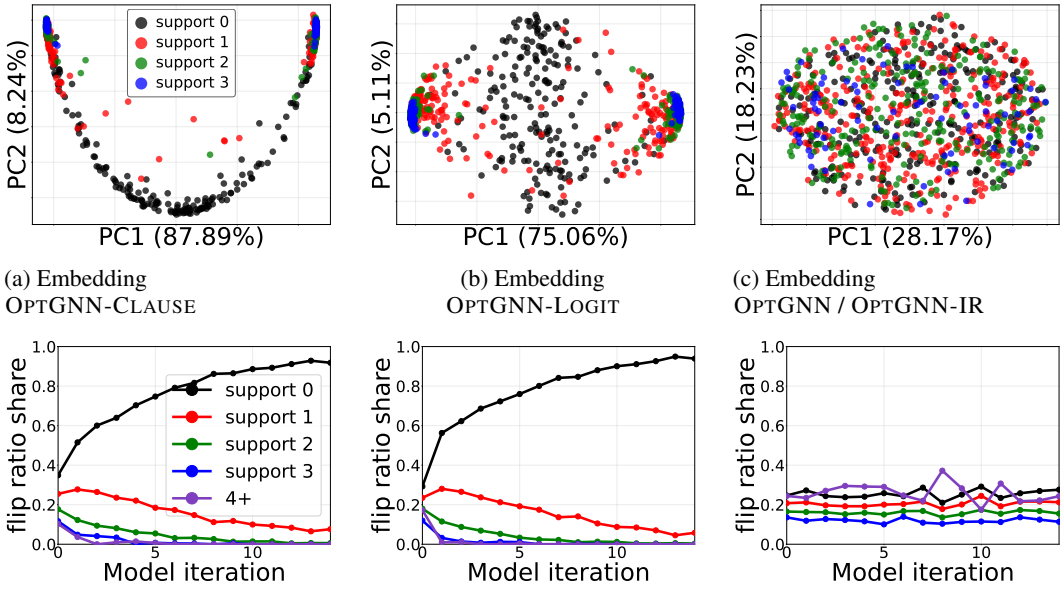
Representation: emergence of confidence. We next examine whether the performance transition corresponds to a change in the variable representations. For each model, we analyze the final variable embeddings and group variables by support, our SAT-specific measure of confidence. The final assignment is extracted from the model output in the usual way: by taking the sign of the continuous prediction for the SDP-style variants, and by thresholding the sigmoid/logit prediction for the logit-based variants. We then use PCA for visualization. Across all variants, PC1 separates the two Boolean sides of the assignment: variables on one side correspond to one truth value, and those on the other to the opposite. Thus, all variants learn a coarse assignment concept. The key question is whether they also organize *confidence*: whether variables with different support levels become geometrically distinguishable in the learned representation space.

Figure 2 and Table 2 show this progression. The table quantifies support organization by training linear SVMs to separate adjacent support levels $(0, 1)$, $(1, 2)$, and $(2, 3)$ in the learned representation space, using 5-fold cross-validation per instance. We report Macro-F1 because adjacent support classes are imbalanced, especially in sparse regimes.

In SAT, for a fixed variable in a 3-literal clause, exactly one of the $2^3 = 8$ local truth assignments makes that variable the unique satisfying literal. Since $m = cn$ clauses contain $3m$ literal occurrences, each variable appears in $3c$ clauses on average. The expected number of support clauses per variable is therefore $3c/8$. This gives a simple regime distinction: for $c < 8/3 \approx 2.67$, the expected support is below 1, so the signal is sparse; for larger c, support becomes a more frequently expressed property that can organize the learned representations.

For NAE-SAT, the analogous support event is that the variable is the "odd one out" in its clause: its literal is the only true literal or the only false literal. For a fixed variable in a 3-literal clause, this occurs in two of the $2^3 = 8$ local truth assignments. Thus, the expected NAE-support per variable is $3c \cdot 2/8 = 3c/4$, crossing 1 already at $c = 4/3$. Hence, support is expected to appear at lower densities in NAE-SAT than in SAT.

The empirical ordering in Table 2 and Table 3 matches this prediction for SAT. At low densities, all models show weak support structure. OPTGNN and OPTGNN-IR remain near chance across all densities, indicating that iterative refinement alone does not organize support. In contrast, OPTGNN-



(a) Embedding OPTGNN-CLAUSE

(b) Embedding OPTGNN-LOGIT

(c) Embedding OPTGNN / OPTGNN-IR

(d) Flip dynamics OPTGNN-CLAUSE

(e) Flip dynamics OPTGNN-LOGIT

(f) Flip dynamics OPTGNN / OPTGNN-IR

Figure 2: Representation and dynamics across model variants on $n = 1000$, $c = 3.7$ SAT instances. Top row: PCA projections of final variable embeddings colored by support; PC1/PC2 labels include explained variance. Bottom row: support-conditioned flip dynamics, averaged over 100 instances. OPTGNN and OPTGNN-IR weakly represent support and still flip high-support variables. OPTGNN-LOGIT and OPTGNN-CLAUSE show similar support-guided flip dynamics, but OPTGNN-CLAUSE organizes support more clearly in embedding space.

Table 2: SVM separability of support representations across model variants and clause-density regimes. For each $n = 1000$ SAT instance and each valid adjacent support pair $(0, 1)$, $(1, 2)$, and $(2, 3)$, we train a linear SVM using 5-fold cross-validation in the learned representation space. Each cell reports Macro-F1 $\pm$ standard deviation, averaged over the clause densities in the corresponding regime. Within each density, Macro-F1 is first averaged over folds and then over instances, and the standard deviation is computed across instances. We report Macro-F1 rather than accuracy because adjacent support levels are often imbalanced. Higher values indicate clearer geometric organization by support. Greyed cells indicate regimes where the expected support size is less than 1.

Pair	Model	$c = 0.5$–1.0	$c = 2.0$–2.5	$c = 3.0$–3.5	$c = 3.7$–4.3
$(0, 1)$	OPTGNN	.46±.02	.50±.02	.50±.03	.49±.03
	OPTGNN-IR	.48±.02	.51±.02	.50±.03	.50±.02
	OPTGNN-LOGIT	.71±.04	.78±.02	.74±.02	.68±.03
	OPTGNN-CLAUSE	**.84±.03**	**.88±.01**	**.92±.01**	**.93±.01**
$(1, 2)$	OPTGNN	.46±.05	.47±.03	.49±.03	.49±.03
	OPTGNN-IR	.45±.02	.48±.03	.50±.03	.50±.03
	OPTGNN-LOGIT	.53±.05	.59±.03	.65±.03	.68±.02
	OPTGNN-CLAUSE	.54±.03	.72±.02	.81±.02	.87±.02
$(2, 3)$	OPTGNN	.42±.04	.47±.05	.48±.04	.48±.04
	OPTGNN-IR	.54±.07	.46±.05	.47±.04	.48±.03
	OPTGNN-LOGIT	.55±.08	.52±.03	.57±.03	.60±.03
	OPTGNN-CLAUSE	.55±.04	.59±.05	.70±.03	.77±.03

LOGIT shows a clear rise in adjacent-support separability, and OPTGNN-CLAUSE improves further and is consistently strongest. The corresponding NAE-SAT results in Table 3 show the same qualitative pattern, with support structure appearing at lower densities, consistent with the higher expected support rate in NAE-SAT. Figure 2 complements Table 2 by visualizing how support is encoded along PC1: in the variants that solve, higher PC1 values typically correspond to higher support, while in the variants that do not, no such alignment is visible.

Dynamics: using confidence. Representation alone is not enough; the model must also use the concept during inference. In SAT, we measure this through support-conditioned flip dynamics: we track which variables flip across iterations as a function of their support. The expected behavior is simple: low-support variables should be revised first, because changing them disrupts few satisfied clauses, whereas high-support variables should be preserved because they uniquely sustain many constraints. This mirrors local-search heuristics such as WalkSAT, which revise unstable variables while avoiding damage to stable parts of the assignment Selman et al. [1993], Hoos and Stützle [2000], Semerjian and Monasson [2003]. It is also consistent with Shoham et al. [2024], where support was identified as the guiding motif behind successful neural SAT solving.

The flip dynamics in Figure 2 follow the same broad progression as the representation analysis, and the same qualitative behavior is observed across the evaluated c values. OPTGNN and OPTGNN-IR behave similarly: both continue to flip high-support variables with relatively high probability, indicating that neither model reliably uses support to protect confident parts of the assignment. Thus, recurrence alone does not fully utilize support, either as a representation-level confidence signal or as a guide for selective refinement. By contrast, OPTGNN-LOGIT and OPTGNN-CLAUSE both exhibit the desired support-guided dynamics: flips are concentrated on low-support variables, while high-support variables are largely preserved. Their main difference is representational rather than dynamical. As shown in Figure 2 and Table 2, OPTGNN-CLAUSE makes support more clearly accessible in the embedding space, whereas OPTGNN-LOGIT uses a similar support-guided flipping pattern but with weaker geometric organization of the support levels. Similarly, Table 3 in the Appendix reports the corresponding SVM separability results for NAE-SAT.

5 Discussion

Our results show that the transition from approximation to solving coincides with a qualitative change in the GNN's latent representation. Across the controlled OPTGNN variants, strong approximation appears early, but reliable solving emerges only when the model learns a coherent confidence signal, instantiated in SAT as *support*. This suggests that solving is not merely better relaxed optimization, but depends on solver-relevant concepts.

Importantly, confidence is not unique to SAT. An analogous notion appears in graph coloring, where the confidence of a vertex assignment can be captured by its minimum degree into each of the other color classes, a quantity closely related to the structural conditions used in classical coloring algorithms such as Alon–Kahale Alon and Kahale [1994]. This points to a broader hypothesis: many neural solvers may succeed precisely when they learn problem-specific notions of confidence that guide which variables should remain fixed and which should be revised.

Several directions remain open. OPTGNN also studies problems such as Vertex Cover, which differ from the cut-based or partition-based CSPs considered here. In Vertex Cover, the solution is defined by membership in a selected set rather than by assigning variables to competing partitions, and it is unclear what combinatorial property should play the role of confidence. Identifying the learned concepts in such settings is an important direction for future work. A related question is how to extend this view to pure optimization problems, such as Max-Cut, where there is generally no exact satisfying assignment to serve as a target solution.

Our study also has limitations. First, we focus primarily on random instances, where density provides a clean control of difficulty; understanding whether the same concepts emerge on industrial, structured, or benchmark SAT distributions remains open. Second, our analysis is restricted to a small family of GNN architectures derived from OPTGNN; broader architectural classes may learn different concepts or encode confidence in different ways. Third, while our representation analysis reveals strong correlations between confidence formation and solving, fully characterizing the causal mechanisms that force such concepts to emerge remains an important theoretical challenge.

References

Dimitris Achlioptas and Amin Coja-Oghlan. Algorithmic barriers from phase transitions. In *49th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2008, Philadelphia, PA, USA, October 25-28, 2008*, pages 793–802. IEEE Computer Society, 2008. doi: 10.1109/FOCS.2008.11. URL https://doi.org/10.1109/FOCS.2008.11.

Dimitris Achlioptas and Federico Ricci-Tersenghi. On the solution-space geometry of random constraint satisfaction problems. In *Proceedings of the 38th ACM Symposium on Theory of Computing*, pages 130–139, 2006.

Dimitris Achlioptas, Arthur D. Chtcherba, Gabriel Istrate, and Cristopher Moore. The phase transition in 1-in-k SAT and NAE 3-sat. In S. Rao Kosaraju, editor, *Proceedings of the Twelfth Annual Symposium on Discrete Algorithms, January 7-9, 2001, Washington, DC, USA*, pages 721–722. ACM/SIAM, 2001. URL http://dl.acm.org/citation.cfm?id=365411.365760.

Dimitris Achlioptas, Assaf Naor, and Yuval Peres. Rigorous location of phase transitions in hard optimization problems. *Nature*, 435(7043):759–764, 2005.

Noga Alon and Nabil Kahale. A spectral technique for coloring random 3-colorable graphs (preliminary version). In *Proceedings of the twenty-sixth annual ACM symposium on Theory of Computing*, pages 346–355, 1994.

Alfredo Braunstein, Marc Mézard, and Riccardo Zecchina. Survey propagation: An algorithm for satisfiability. *Random Structures & Algorithms*, 27(2):201–226, 2005.

Alfredo Braunstein, Marc Mézard, Martin Weigt, and Riccardo Zecchina. Constraint satisfaction by survey propagation. In Allon G. Percus, Gabriel Istrate, and Cristopher Moore, editors, *Computational Complexity and Statistical Physics*, Santa Fe Institute Studies in the Sciences of Complexity, pages 107–124. Oxford University Press, 2006.

Stephen A. Cook. The complexity of theorem-proving procedures. In Michael A. Harrison, Ranan B. Banerji, and Jeffrey D. Ullman, editors, *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing, May 3-5, 1971, Shaker Heights, Ohio, USA*, pages 151–158. ACM, 1971. doi: 10.1145/800157.805047. URL https://doi.org/10.1145/800157.805047.

Abraham Flaxman. A spectral technique for random satisfiable 3cnf formulas. *Random Struct. Algorithms*, 32 (4):519–534, 2008. doi: 10.1002/RSA.20213. URL https://doi.org/10.1002/rsa.20213.

Ehud Friedgut, Jean Bourgain, et al. Sharp thresholds of graph properties, and the k-SAT problem. *Journal of the American Mathematical Society*, 12(4):1017–1054, 1999.

Justin et al. Gilmer. Neural message passing for quantum chemistry. In *ICML*, 2017.

Johan Håstad. Some optimal inapproximability results. *Journal of the ACM*, 48(4):798–859, 2001.

Holger H. Hoos and Thomas Stützle. Local search algorithms for SAT: an empirical evaluation. *J. Autom. Reason.*, 24(4):421–481, 2000. doi: 10.1023/A:1006350622830. URL https://doi.org/10.1023/A:1006350622830.

Henrique Lemos, Marcelo O. R. Prates, Pedro H. C. Avelar, and Luís C. Lamb. Graph colouring meets deep learning: Effective graph neural network models for combinatorial problems. *CoRR*, abs/1903.04598, 2019. URL http://arxiv.org/abs/1903.04598.

Andrea Lincoln and Adam Yedidia. Faster random k-cnf satisfiability. In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *47th International Colloquium on Automata, Languages, and Programming, ICALP 2020, July 8-11, 2020, Saarbrücken, Germany (Virtual Conference)*, volume 168 of *LIPIcs*, pages 78:1–78:12. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. doi: 10.4230/LIPIcs.ICALP.2020.78. URL https://doi.org/10.4230/LIPIcs.ICALP.2020.78.

Stephan Mertens, Marc Mézard, and Riccardo Zecchina. Threshold values of random k-sat from the cavity method. *Random Structures & Algorithms*, 28(3):340–373, 2006.

Marc Mézard and Riccardo Zecchina. Random k-satisfiability problem: From an analytic solution to an efficient algorithm. *Physical Review E*, 66(5):056126, 2002.

Eugene Nudelman, Kevin Leyton-Brown, Holger H Hoos, Alex Devkar, and Yoav Shoham. Understanding random sat: Beyond the clauses-to-variables ratio. In *Principles and Practice of Constraint Programming–CP 2004: 10th International Conference, CP 2004, Toronto, Canada, September 27-October 1, 2004. Proceedings 10*, pages 438–452. Springer, 2004.

Judea Pearl. *Probabilistic reasoning in intelligent systems: networks of plausible inference*. Representation and Reasoning Series. Morgan Kaufmann, 1988. ISBN 9781558604797. URL http://books.google.de/books?id=AvNID7LyMusC.

Sakari Seitz, Mikko Alava, and Pekka Orponen. Threshold behaviour of walksat and focused metropolis search on random 3-satisfiability. In *International Conference on Theory and Applications of Satisfiability Testing*, pages 475–481. Springer, 2005.

Bart Selman, Henry A. Kautz, and Bram Cohen. Local search strategies for satisfiability testing. In David S. Johnson and Michael A. Trick, editors, *Cliques, Coloring, and Satisfiability, Proceedings of a DIMACS Workshop, New Brunswick, New Jersey, USA, October 11-13, 1993*, volume 26 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 521–531. DIMACS/AMS, 1993. doi: 10.1090/dimacs/026/25. URL https://doi.org/10.1090/dimacs/026/25.

Daniel Selsam, Matthew Lamm, Benedikt Bünz, Percy Liang, Leonardo de Moura, and David L. Dill. Learning a SAT solver from single-bit supervision. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL https://openreview.net/forum?id=HJMC_iA5tm.

Guilhem Semerjian and Rémi Monasson. Relaxation and metastability in a local search procedure for the random satisfiability problem. *Physical Review E*, 67(6):066103, 2003.

Elad Shoham, Hadar Cohen, Khalil Wattad, Havana Rika, and Dan Vilenchik. Concept learning in the wild: Towards algorithmic understanding of neural networks, 2024.

Elad Shoham, Omri Haber, Havana Rika, and Dan Vilenchik. Learning to rank: How gnns solve max-clique and sparse PCA. In Sven Koenig, Chad Jenkins, and Matthew E. Taylor, editors, *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20-27, 2026*, pages 25401–25409. AAAI Press, 2026. doi: 10.1609/AAAI.V40I30.39734. URL https://doi.org/10.1609/aaai.v40i30.39734.

Jan Tönshoff, Martin Ritzert, Hinrikus Wolf, and Martin Grohe. Graph Neural Networks for Maximum Constraint Satisfaction. *Frontiers in Artificial Intelligence*, 3, 2021.

Morris Yau, Nikolaos Karalias, Eric Lu, Jessica Xu, and Stefanie Jegelka. Are graph neural networks optimal approximation algorithms? In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/85db52cc08c5e00cfb1d216b1c85ba35-Abstract-Conference.html.

Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. Graph neural networks: A review of methods and applications. *AI open*, 1:57–81, 2020.

A Support as a signal

A.1 Support induces stabilization through the loss gradient

Insight 1 (The SAT loss induces a strict ordering of relaxed clause patterns)**.** *Consider the relaxed SAT clause-unsatisfaction loss*

$$u_C^{\mathrm{SAT}}(x) = \prod_{i \in C^+} (1 - x_i) \prod_{i \in C^-} x_i.$$

For an ϵ-relaxed Boolean assignment, where each true literal has false-indicator ϵ and each false literal has false-indicator $1 - \epsilon$, with $0 < \epsilon < 1/2$, the clause patterns satisfy

$$u_C(TTT) < u_C(TTF) < u_C(TFF) < u_C(FFF).$$

Thus, among satisfied clauses, uniquely satisfied clauses are assigned the largest relaxed unsatisfaction value. This makes variables that uniquely satisfy many clauses locally distinguishable as confidence-critical variables.

Proof. For a positive literal $i \in C^+$, the literal is false when $x_i = 0$, so its relaxed false-indicator is $1-x_i$. For a negative literal $i \in C^-$, the literal $\neg x_i$ is false when $x_i = 1$, so its relaxed false-indicator is x_i. Therefore,

$$u_C^{\mathrm{SAT}}(x) = \prod_{i \in C^+} (1 - x_i) \prod_{i \in C^-} x_i.$$

Under an ϵ-relaxed Boolean assignment, every true literal has false-indicator ϵ, and every false literal has false-indicator $1 - \epsilon$, regardless of whether the literal is positive or negated. Hence, for a 3-literal clause,

$$u_C(TTT) = \epsilon^3, \qquad u_C(TTF) = \epsilon^2(1-\epsilon), \qquad u_C(TFF) = \epsilon(1-\epsilon)^2, \qquad u_C(FFF) = (1-\epsilon)^3.$$

Since $0 < \epsilon < 1/2$, we have $\epsilon < 1 - \epsilon$. Therefore,

$$\epsilon^3 < \epsilon^2(1-\epsilon) < \epsilon(1-\epsilon)^2 < (1-\epsilon)^3,$$

and so

$$u_C(TTT) < u_C(TTF) < u_C(TFF) < u_C(FFF).$$

Thus, among satisfied clauses, the uniquely satisfied pattern TFF is closest to violation under the relaxed loss. Aggregated over clauses, variables that uniquely satisfy many clauses repeatedly participate in these high-risk satisfied clauses, providing a local confidence signal.

Moreover, this high-risk status has a stabilizing derivative. In the positive case, $TFF = (1 - \epsilon, \epsilon, \epsilon)$ gives

$$\frac{\partial u_C}{\partial x_T} = -(1-\epsilon)^2,$$

so gradient descent increases the unique true literal and moves the clause away from the violated FFF pattern. Thus, TFF is not only closest to violation in value; it also produces a direct local push preserving the literal that prevents violation.

□

Insight 2 (The SDP loss mixes support with vector-moment noise)**.** *The SDP-style clause objective does not expose support as a clean pattern-level signal. Its value is determined by continuous inner-product moments of the lifted vector solution, not only by the rounded Boolean assignment. Consequently, two rounded patterns such as TFF and TTF need not obey a universal SDP-value ordering. Thus, any support-related signal in the SDP loss is mixed with geometry-dependent variation, making it harder for the model to isolate and use.*

Proof. Consider the SDP-style clause contribution

$$\begin{aligned}\Phi_C(V) = \frac{1}{8}\Bigg(7 &- \tau_i\tau_j\tau_k \frac{1}{3}\left[\langle v_i, v_{jk}\rangle + \langle v_j, v_{ik}\rangle + \langle v_k, v_{ij}\rangle\right] \\ &- \tau_i\tau_j \frac{1}{2}\left[\langle v_i, v_j\rangle + \langle v_{ij}, v_\emptyset\rangle\right] - \tau_i\tau_k \frac{1}{2}\left[\langle v_i, v_k\rangle + \langle v_{ik}, v_\emptyset\rangle\right] \\ &- \tau_j\tau_k \frac{1}{2}\left[\langle v_j, v_k\rangle + \langle v_{jk}, v_\emptyset\rangle\right] - \tau_i\langle v_i, v_\emptyset\rangle - \tau_j\langle v_j, v_\emptyset\rangle - \tau_k\langle v_k, v_\emptyset\rangle\Bigg).\end{aligned} \tag{5}$$

Here, $\tau_i, \tau_j, \tau_k \in \{-1, +1\}$ encode the signs of the literals in clause C. The vector v_i is the SDP vector associated with variable x_i, while v_{ij}, v_{ik}, and v_{jk} are lifted SDP vectors associated with the corresponding variable pairs. The vector $v_\emptyset$ is the distinguished reference vector, representing the constant or empty monomial. Thus, the terms in Equation (5) are inner-product moments of the lifted SDP representation: unary moments such as $\langle v_i, v_\emptyset\rangle$, pairwise moments such as $\langle v_i, v_j\rangle$ and $\langle v_{ij}, v_\emptyset\rangle$, and higher-order lifted moments such as $\langle v_i, v_{jk}\rangle$.

Define the scalar vector moments

$$a_i(V) = -\tau_r\langle v_i, v_\emptyset\rangle,$$

$$b_{ij}(V) = \tau_r\tau_s \frac{1}{2}\left[\langle v_i, v_j\rangle + \langle v_{ij}, v_\emptyset\rangle\right],$$

and

$$c_{ijk}(V) = -\tau_i\tau_j\tau_k \frac{1}{3}\left[\langle v_i, v_{jk}\rangle + \langle v_j, v_{ik}\rangle + \langle v_k, v_{ij}\rangle\right].$$

Then

$$\Phi_C(V) = \frac{1}{8}\left(7 + a_i(V) + a_j(V) + a_k(V) - b_{ij}(V) - b_{ik}(V) - b_{jk}(V) + c_{ijk}(V)\right).$$

The clause value is therefore a function of scalar inner-product moments of the SDP vectors. These moments are continuous geometric quantities; they are not Boolean truth values and are not determined by the rounded Boolean pattern alone.

To make this explicit, compare two rounded satisfying patterns that share literals i and k, but differ in the middle literal:

$$TFF = (i : T,\ j : F,\ k : F), \qquad TTF = (i : T,\ j' : T,\ k : F).$$

In TFF, literal i uniquely satisfies the clause. In TTF, the clause is redundantly satisfied. Using the scalar moment notation above, the corresponding SDP contributions are

$$\Phi_C(TFF; V) = \frac{1}{8}\left(7 + a_i - a_j - a_k + b_{ij} + b_{ik} - b_{jk} + c_{ijk}\right),$$

and

$$\Phi_{C'}(TTF; V) = \frac{1}{8}\left(7 + a_i + a_{j'} - a_k - b_{ij'} + b_{ik} + b_{j'k} - c_{ij'k}\right).$$

Subtracting gives

$$\Phi_C(TFF; V) - \Phi_{C'}(TTF; V) = \frac{1}{8}\left(-a_j - a_{j'} + b_{ij} + b_{ij'} - b_{jk} - b_{j'k} + c_{ijk} + c_{ij'k}\right).$$

The shared terms a_i, a_k, and b_{ik} cancel. The remaining terms depend on the non-shared literals j and j', and on their SDP moments with i and k. These moments need not be equal, and their comparison is not fixed by the rounded support pattern.

Therefore, the rounded fact that TFF is uniquely supported while TTF is redundantly satisfied does not imply either

$$\Phi_C(TFF; V) \geq \Phi_{C'}(TTF; V)$$

or

$$\Phi_C(TFF; V) \leq \Phi_{C'}(TTF; V)$$

for all relaxed SDP configurations. A universal ordering would require additional assumptions on the SDP moments.

The consistency and unit-norm penalties used in Yau et al. [2024] restrict the feasible vector geometry, but they do not make the SDP clause value a function only of the rounded Boolean pattern. Even under strong consistency, the value remains determined by continuous moment-like quantities of the vector solution.

Hence the SDP loss may contain information correlated with support, but it does not expose support as a clean local signal in the same way as the logit loss. The support-related effect is mixed with geometry-dependent variation in the vector moments, which acts as noise from the perspective of support detection. This makes the support signal harder for the model to utilize. □

B NAE tables

Table 3: SVM separability of support representations for NAE-SAT. For each $n = 1000$ NAE-SAT instance and each valid adjacent support pair $(0, 1)$, $(1, 2)$, and $(2, 3)$, we train a linear SVM using 5-fold cross-validation in the learned representation space. Each cell reports Macro-F1 $\pm$ standard deviation, where Macro-F1 is first averaged over folds and then over instances, and the standard deviation is computed across instances. Higher values indicate clearer geometric organization by support.

c	Model	$(0, 1)$	$(1, 2)$	$(2, 3)$
0.5	OPTGNN-IR-NAE	.46±.02	.44±.04	–
	OPTGNN-LOGIT-NAE	.73±.04	.59±.05	.56±.09
	OPTGNN-CLAUSE-NAE	**.91±.03**	.76±.06	.55±.10
1.0	OPTGNN-IR-NAE	.45±.02	.47±.04	.48±.07
	OPTGNN-LOGIT-NAE	.74±.02	.59±.04	.54±.06
	OPTGNN-CLAUSE-NAE	**.92±.02**	**.82±.03**	.61±.07
1.5	OPTGNN-IR-NAE	.48±.02	.45±.04	.46±.06
	OPTGNN-LOGIT-NAE	.70±.02	.59±.03	.53±.06
	OPTGNN-CLAUSE-NAE	**.95±.01**	**.87±.02**	**.70±.06**
1.8	OPTGNN-IR-NAE	.48±.02	.46±.04	.44±.07
	OPTGNN-LOGIT-NAE	.67±.02	.59±.03	.53±.05
	OPTGNN-CLAUSE-NAE	**.94±.02**	**.87±.03**	**.72±.06**
2.0	OPTGNN-IR-NAE	.49±.02	.47±.03	.46±.06
	OPTGNN-LOGIT-NAE	.64±.02	.59±.03	.54±.05
	OPTGNN-CLAUSE-NAE	**.93±.01**	**.86±.02**	**.73±.04**
2.1	OPTGNN-IR-NAE	.49±.02	.48±.02	.48±.05
	OPTGNN-LOGIT-NAE	.63±.02	.60±.03	.55±.05
	OPTGNN-CLAUSE-NAE	**.93±.01**	**.85±.02**	**.73±.04**
2.5	OPTGNN-IR-NAE	.51±.02	.48±.03	.47±.04
	OPTGNN-LOGIT-NAE	.59±.02	.58±.03	.54±.04
	OPTGNN-CLAUSE-NAE	**.91±.01**	**.84±.02**	**.73±.04**

Table 4: SAT and NAE results by clause density for $n = 1000$, over 100 instances per density. Each cell reports SAT found | NAE found | SAT approximation | NAE approximation.

c	Model	SAT found	NAE found	SAT approx.	NAE approx.
0.5	OPTGNN-IR-NAE	0.59	0.00	0.99	0.82
	OPTGNN-CLAUSE	1.00	0.00	1.00	0.66
	OPTGNN-LOGIT-NAE	0.78	0.21	1.00	1.00
	OPTGNN-CLAUSE-NAE	0.83	0.09	0.99	0.99
1.0	OPTGNN-IR-NAE	1.00	0.00	1.00	0.95
	OPTGNN-CLAUSE	1.00	0.00	1.00	0.77
	OPTGNN-LOGIT-NAE	1.00	0.81	1.00	1.00
	OPTGNN-CLAUSE-NAE	1.00	0.98	1.00	1.00
1.5	OPTGNN-IR-NAE	0.26	0.00	0.99	0.92
	OPTGNN-CLAUSE	1.00	0.00	1.00	0.82
	OPTGNN-LOGIT-NAE	0.97	0.11	1.00	1.00
	OPTGNN-CLAUSE-NAE	1.00	1.00	1.00	1.00
1.8	OPTGNN-IR-NAE	0.00	0.00	0.99	0.92
	OPTGNN-CLAUSE	1.00	0.00	1.00	0.83
	OPTGNN-LOGIT-NAE	0.07	0.00	0.99	0.99
	OPTGNN-CLAUSE-NAE	0.86	0.23	0.99	0.99
2.0	OPTGNN-IR-NAE	0.00	0.00	0.99	0.92
	OPTGNN-CLAUSE	1.00	0.00	1.00	0.84
	OPTGNN-LOGIT-NAE	0.01	0.00	0.99	0.99
	OPTGNN-CLAUSE-NAE	0.07	0.00	0.99	0.99
2.1	OPTGNN-IR-NAE	0.00	0.00	0.99	0.92
	OPTGNN-CLAUSE	1.00	0.00	1.00	0.84
	OPTGNN-LOGIT-NAE	0.00	0.00	0.98	0.98
	OPTGNN-CLAUSE-NAE	0.01	0.00	0.99	0.99
2.5	OPTGNN-IR-NAE	0.00	0.00	0.99	0.92
	OPTGNN-CLAUSE	1.00	0.00	1.00	0.85
	OPTGNN-LOGIT-NAE	0.00	0.00	0.98	0.98
	OPTGNN-CLAUSE-NAE	0.00	0.00	0.98	0.98

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes].

   Justification: The abstract and introduction accurately state the paper's main claims: relaxation-based neural CSP solvers can achieve high approximation ratios while still failing to produce exact satisfying assignments; the transition from approximation to solving is associated with learning and using a solver-relevant confidence concept, instantiated in SAT as support; and controlled architectural changes to OPTGNN, especially logit-space supervision and explicit constraint-level aggregation, expose and organize this signal. These claims are evaluated through the controlled OPTGNN variants, approximation-versus-solving results, representation analysis, support-separability probes, flip-dynamics analysis, and NAE-SAT experiments.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper discusses the main limitations of the work in the discussion section. It states that the experiments focus primarily on random instances, where clause density provides a controlled notion of difficulty; that the analysis is restricted to a small family of OPTGNN-derived architectures; and that the observed link between confidence formation and solving is mechanistic and correlational rather than a complete causal characterization. The paper also notes that support is the concrete confidence concept studied for SAT, while identifying the corresponding concepts in other CSPs or optimization problems remains an open direction.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.

- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: The paper includes theoretical support for the main mechanistic claims in Appendix A. In particular, it formalizes why the SAT logit-space clause-unsatisfaction loss exposes support as a local confidence signal, and why the SDP-style clause objective does not impose a universal ordering over support patterns because its value depends on continuous vector moments. The assumptions needed for these arguments are stated explicitly, including the near-discrete ϵ-relaxed Boolean assignment condition for the SAT loss analysis and the lifted SDP moment notation for the SDP comparison. The appendix provides the corresponding derivations and proofs, and the main text references these insights when motivating the role of the logit loss and support-based stabilization.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper provides the information needed to reproduce the main experimental claims. It identifies the released OptGNN implementation as the implementation base, states that unchanged settings follow that implementation, and specifies the controlled model modifications studied. It also reports the data generation procedure, training and evaluation instance sizes, clause-density ranges, number of test instances, evaluation metrics, decoding rules, representation-probing protocol, support-conditioned flip-dynamics protocol, Hamming-distance diagnostic, inference settings, hidden dimension, software environment, hardware environment, and number of independent runs. Together, these details provide a reasonable path for reproducing the main approximation-versus-solving, representation, and dynamics results.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.

- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: The paper does not currently release a separate codebase or supplemental reproduction package. The experiments are based on the publicly released OptGNN implementation, which is cited in the paper, and the paper describes the controlled modifications, synthetic data generation procedure, evaluation protocol, and metrics used in our experiments. Since the data are synthetically generated rather than collected from an external dataset, no separate dataset release is required. However, because the modified implementation, exact scripts, and run commands are not currently provided as open-access supplemental material, we answer [No]. The paper nevertheless provides the main implementation base and methodological details needed to guide reproduction of the reported results. If required for publication or reproducibility review, we will release an implementation or patch containing the modifications needed to reproduce the main experiments.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.

- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper specifies the experimental details needed to understand and interpret the results. It describes the controlled OPTGNN variants, the data generation procedure for SAT and NAE-SAT, the training and evaluation instance sizes, the clause-density ranges, the number of test instances per density, the evaluation metrics, the decoding rules, the representation-probing protocol, and the flip-dynamics protocol. It also reports key implementation details such as inference iterations, hidden dimension, software environment, hardware environment, and number of independent runs. Low-level settings not changed by this work follow the released OPTGNN implementation, which is cited as the implementation base.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: The paper reports variability information for the main experimental results. For the SAT solving and approximation results, the table caption states that the standard deviation across test instances was below 0.01 in all reported settings and was therefore omitted for readability. The results section also reports that each model was trained and evaluated over 10 independent runs, with reported metrics varying by standard deviation at most 0.02 across runs. For the representation analysis, the SVM support-separability tables report Macro-F1 scores with standard deviations, computed across instances after 5-fold cross-validation. The paper does not rely on formal hypothesis tests; the reported variability is used to show that the qualitative ordering across model variants is stable.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.

- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The paper reports the compute resources used for the experiments in the Results section, including the workstation CPU, RAM, GPU, software environment, and implementation base. It also reports the number of independent runs and the main inference settings used for evaluation. Together with the reported runtime information, these details provide sufficient compute context to understand the scale of the experiments and reproduce the main results.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work is methodological and uses synthetically generated combinatorial optimization instances rather than human-subject data or personally identifiable information. The paper reports the experimental setting, compute resources, limitations, and reproducibility-relevant details, and we do not identify any foreseeable ethical concerns such as privacy, discrimination, surveillance, deception, or safety harms.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: The work is foundational and methodological, focusing on representation and concept emergence in neural solvers for synthetic combinatorial optimization instances. It does not introduce a deployed system, human-facing application, dataset involving people, or a capability with a direct path to societal harm. Potential downstream benefits include better understanding and evaluation of neural combinatorial solvers, but these impacts are indirect and application-dependent.

    Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release data or models with a high risk for misuse. The experiments use synthetically generated combinatorial optimization instances and modified neural solver implementations, not scraped datasets, human-subject data, generative models, or dual-use models requiring controlled release. Therefore, no special safeguards for responsible release are applicable.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The paper properly credits the existing OptGNN implementation by citing the original work. The implementation is released under the MIT License, which permits use, modification, and redistribution subject to preserving the copyright and permission notice. Our work builds on that implementation, uses synthetically generated data, and does not rely on external datasets with additional licensing constraints.

    Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: The paper does not release new standalone assets such as a new dataset, benchmark, model checkpoint, or independent software package. The experiments use synthetically generated instances and modifications of the publicly released OptGNN implementation, which is cited and licensed separately.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing, human-subject experiments, participant interaction, or human-generated annotations. All experiments use synthetically generated combinatorial optimization instances.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing, human-subject research, participant interaction, or data collected from people. All experiments use synthetically generated combinatorial optimization instances, so IRB approval or equivalent human-subjects review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: The core method development, experiments, and analysis do not involve LLMs as an important, original, or non-standard component. Any LLM use was limited, if at all, to writing, editing, or formatting assistance and did not affect the methodology, experimental results, scientific claims, or originality of the research.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.